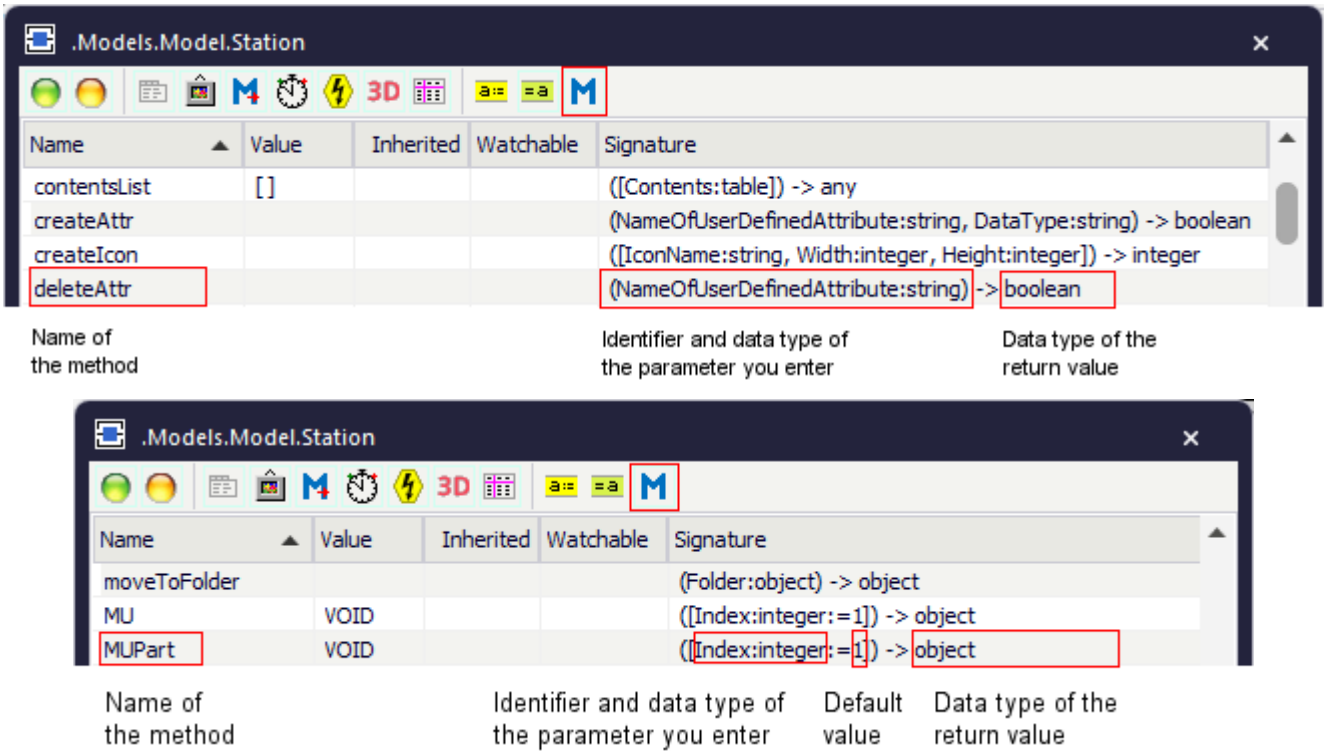
- query information from an object and return a value
- make calculations
- starts one or several actions that control the behavior of the object

We grouped the *methods* in the *Plant Simulation Help* according to their functions.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type

string. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

> **Note**
>
> Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

If the *method* has a return value, the signature shows its data type after the arrow ->, → `boolean` in the example above.

See also

_General Methods

_Methods of Object Icons

_Methods for the Location, Predecessor, and Successor

_Methods for Managing Inheritance Relations

_Methods for Managing Attributes

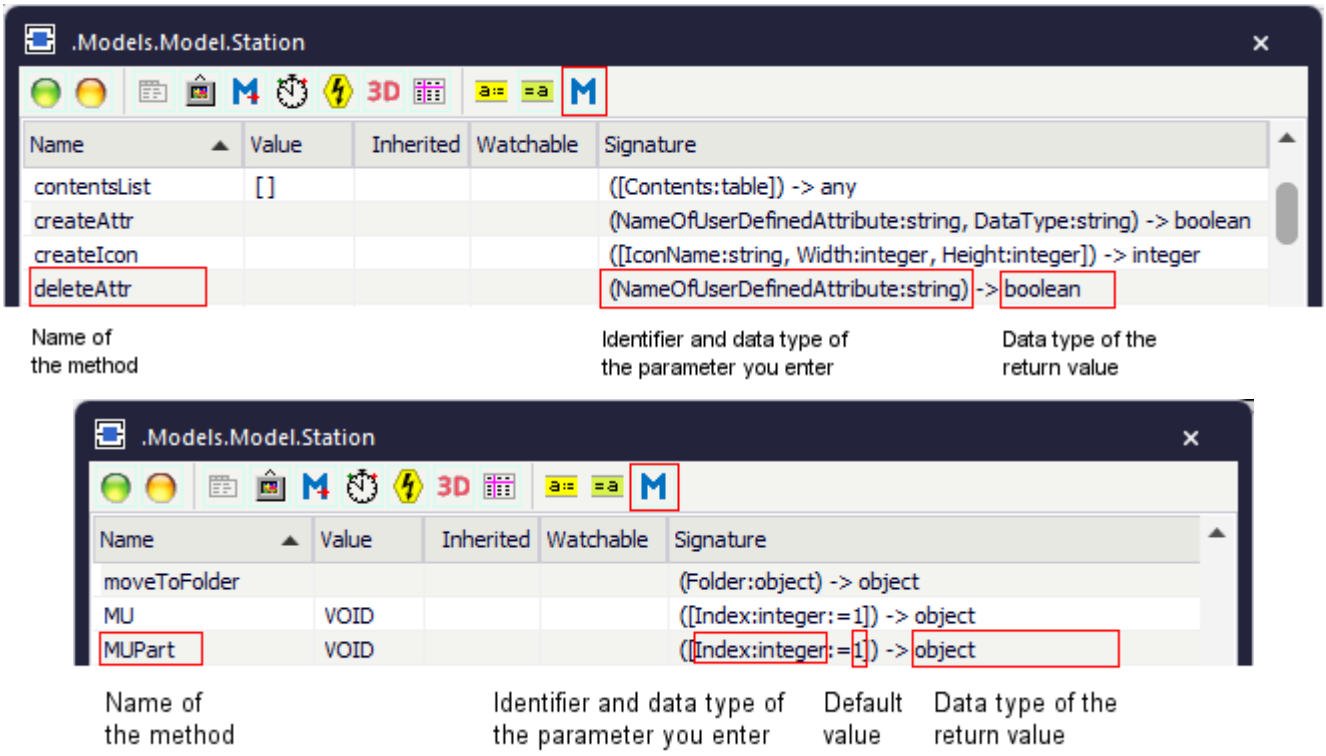
_Methods of User-defined Attributes

General Methods

_General Methods

Most of the objects provide the methods listed in the table of contents to the left.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected **Instance [general description]**.

addObserver [SimTalk]

Adds an observer to the object designated by <Path>.

Type

Method

Syntax

```
<Path>.addObserver(AttributeName:string, Method:object)
```

Parameters

You can specify the following parameters:

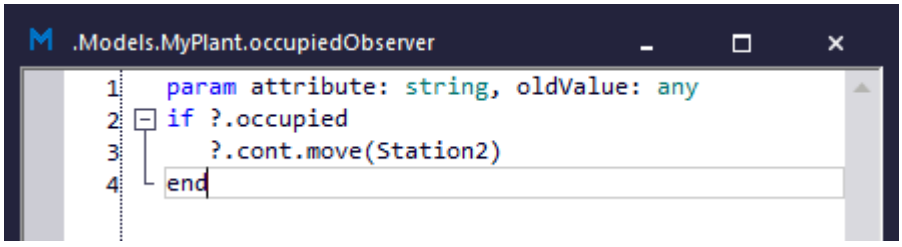
- The parameter *AttributeName* of data type *string* designates the name of the **watchable** *attribute* or *read-only attribute*.
- The parameter *method* of data type *object* designates the *Method* that is going to be called when the value of this *attribute* or *read-only attribute* changes. *Plant Simulation* then executes the actions, which you programmed in this *method*.

  Within the called *Method* you can use the anonymous identifiers **?** and **@** to address the object, whose attribute changed, i.e., the caller of the *Method*.

  Two parameters are passed to the *Method*, which will be executed:

  - The name of the watched *attribute* or the watched *read-only attribute*, whose value changed. This way you can use a single *Method* as the called *Method* for several attributes.
  - The previous value of the watchable *attribute* or the watchable *read-only attribute*. This way you can still access the previous value after the *Method* changed it to the new value.

    The source code for our *occupiedObserver* looks like this:



Example

```
MyStation.addObserver("occupied", &myMethod)
```

See also

Edit Observers [general description]

removeObserver [SimTalk]

removeAllObservers [SimTalk]

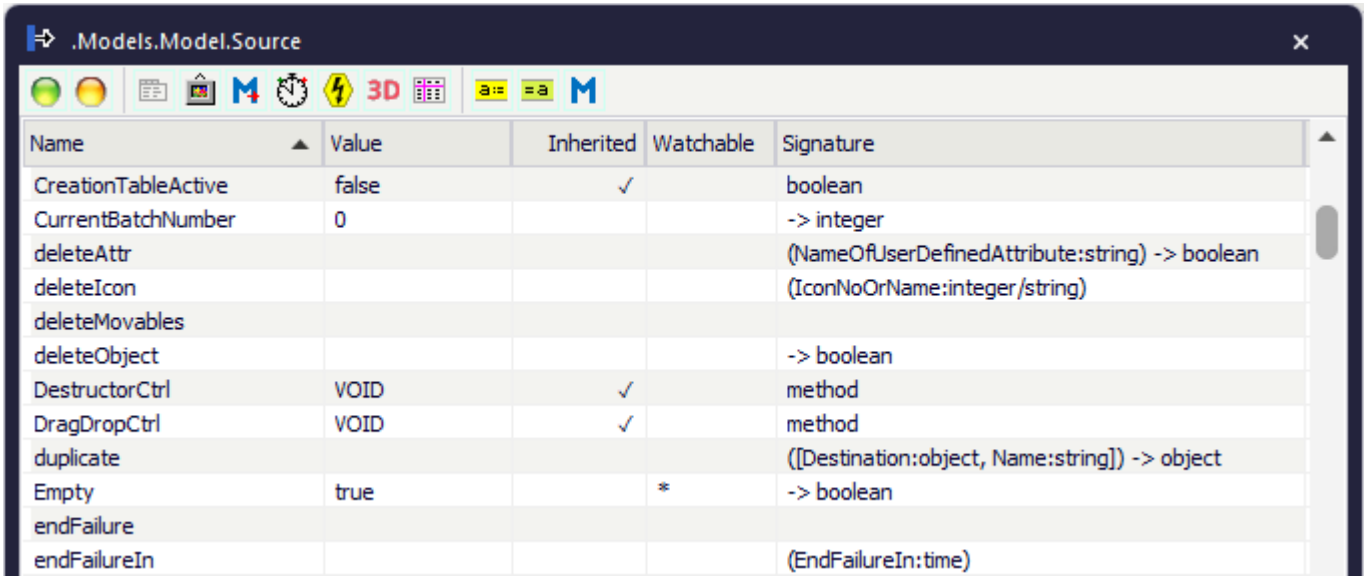
attributeWatchable [SimTalk]

attributeWatchable [SimTalk]

Returns if an *attribute* or *read-only attribute* of the object designated by <Path> is **watchable** (true) or not (false).

Remarks

To check which *attributes* and *read-only attributes* of an object are watchable, you can also select the object and press **F8**. Watchable *attributes* and *read-only attributes* are marked with an asterisk (*****) in the column **Watchable** in the window **Show Attributes and Methods**.



Name	Value	Inherited	Watchable	Signature
CreationTableActive	false	✓		boolean
CurrentBatchNumber	0			-> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean
deleteIcon				(IconNoOrName:integer/string)
deleteMovables				
deleteObject				-> boolean
DestructorCtrl	VOID	✓		method
DragDropCtrl	VOID	✓		method
duplicate				([Destination:object, Name:string]) -> object
Empty	true		*	-> boolean
endFailure				
endFailureIn				(EndFailureIn:time)

Type

Method

Syntax

```
<Path>.attributeWatchable(AttributeName:string) → boolean
```

Parameter

The parameter *AttributeName* of data type *string* designates the *attribute* or *read-only attribute*.

Return Value

The return value has the data type *boolean*.

Example

```
print ParallelStation.attributeWatchable("NumMU")
```

See also

Show Attributes and Methods [described]

Watchable Values

closeDialog [SimTalk] - all objects

Closes the dialog box of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.closeDialog([ApplyChanges:boolean:=true]) → boolean
```

Parameter

The optional parameter *ApplyChanges* of data type *boolean* determines if *Plant Simulation* ignores all changes you made in the dialog box or if it accepts and applies your changes.

Specify `true` to make the object evaluate all settings in the dialog box and apply them as the new values.

Specify `false` to make the object discard your changes.

Default Value of the Parameter

The default value is true.

Return Value

The return value has the data type *boolean*.

Example

```
MyStore.closeDialog            // apply changes
MyTrack.closeDialog(false)     // do not apply changes
```

See also

_3D.closeWindows [SimTalk]

OK button

Cancel button

deleteObject [SimTalk]

Deletes the object designated by <Path>.

Remarks

You cannot delete objects that are currently active, for example *Methods* that are being executed at the moment. For this reason you cannot, for example, delete a *Frame* using the *Constructor method* if the *Constructor method* itself is located within that *Frame*.

> **Note**
> Deleting a class object automatically deletes **all of its instances** without warning.

> **Note**
> To delete a *MU*, we recommend to use the method **delete**. This prevents that you accidentally delete part of your simulation model when you address the wrong object.

Type

Method

Syntax

```
<Path>.deleteObject → boolean
```

Return Value

The return value has the data type *boolean*.

true if the object was deleted successfully.

false if the object was not deleted.

Example

```
.Materialflow.MyStation.deleteObject
// deletes the object from the class library
MyStation.deleteObject
// deletes the object from the Frame
```

See also

Delete [Home ribbon]

delete [SimTalk] - MUs

derive [SimTalk]

Derives the object designated by <Path> in the *Class Library*, i.e., it creates a subclass that inherits its properties from the class of the derived object.

Remarks

Plant Simulation immediately applies all changes in the class to the subclass.

Type

Method

Syntax

```
<Path>.derive([Destination:object, Name:string,
NextRandomSeedValue:integer]) → object
```

Parameters

You can specify the following parameters:

- The optional parameter *Destination* of data type *object* designates the destination at which *Plant Simulation* creates the object.
- The optional parameter *Name* of data type *string* designates the name of the object that will be created. If this *Name* cannot be used because it is not unique, *Plant Simulation* shows an error message.
- The optional parameter *NextRandomSeedValue* of data type *integer* sets which **Random Seed Value** is going to be used next. *Plant Simulation* uses this value instead of the value which you can query with the method *setRandomSeedCounter*.

  The parameter *NextRandomSeedValue* ensures reproducible simulation behavior when you create objects during the simulation.

Return Value

The return value has the data type *object*.

It is the derived object, if it was able to derive the object.

VOID if deriving failed.

Examples

```
var Station := .MaterialFlow.Station.derive
// Creates a new class in the folder MaterialFlow which is derived from the
```

```
object named Station.
Station.Name := “TestCenter”
Station.Coordinate3D := [4.0, 5.0, 0.0]
```

```
var NewVariable : object := .InformationFlow.&Variable.derive
// To derive a Variable, we have to use the &-operator to prevent calling
// the command ‘derive’ for the value of the Variable.
```


```
var NewVariable : object := .InformationFlow.&Variable.derive
// To derive a Variable, we have to use the &-operator to prevent calling
// the command ‘derive’ for the value of the Variable.
```


SimTalk

duplicate [SimTalk]

deleteObject [SimTalk]

RandomSeed [SimTalk] - material flow objects

setRandomSeedCounter [SimTalk]

create [SimTalk] - MUs

See also

Derive

Cut Inheritance

Random Seed Value

duplicate [SimTalk]

Copies the object designated by <Path> in the *Class Library* and creates a new class by duplicating it.

Remarks

Plant Simulation cuts inheritance relations to the original of the duplicated object.

To create the object at a certain position in the *Frame*, you can use the attribute *Coordinate3D*.

Type

Method

Syntax

```
<Path>.duplicate([Destination:object, Name:string]) → object
```

Parameters

You can specify the following parameters:

- The optional parameter *Destination* of data type *object* designates the destination at which *Plant Simulation* creates the object.
- The optional parameter *Name* of data type *string* designates the name of the object that will be created.

Return Value

The return value has the data type *object*.

It is the duplicated object, if it was able to duplicate the object.

VOID if duplicating failed.

Examples

```
.MaterialFlow.Conveyor.duplicate
.MaterialFlow.Conveyor.duplicate(.Models.MyPlant,"MyConveyor")
.InformationFlow.&Variable.duplicate
.InformationFlow.&Method.duplicate
```

```
var
myConveyor:object := .MaterialFlow.Conveyor.duplicate(.Models.Model."MyConve
```

```
yor")
MyConveyor.setPosition := [100, 100] // Sets the pixel coordinates.
```

SimTalk

derive [SimTalk]

deleteObject [SimTalk]

Coordinate3D [SimTalk] - general description

See also

Duplicate

Cut Inheritance

extendPath [SimTalk]

Extends the path of the object designated by <Path>, allowing you to access objects contained within that object.

Type

Method

Syntax

```
<Path>.extendPath(PathExtension:string) → object
```

Parameter

The parameter *PathExtension* of data type *string* designates the path of the object whose path you want to extend.

Return Value

The return value has the data type *object*.

Examples

```
var obj := MyFrame.extendPath("Station3")

if obj /= void
   obj.ProcTime := 63.5
end
```

```
current.extendPath("Station") -- tells if the object named Station
                              -- exists in the Frame current
```

See also

existsObject [SimTalk]

getHTMLCode [SimTalk] - all objects

Returns the HTML code for the object designated by <Path> with statistics which would generate a statistics table in the *HtmlReport*.

Remarks

The syntax is the same as in the **HtmlReport**.

Type

Method

Syntax

```
<Path>.getHTMLCode([Caption:string,
StatisticsType:string,ColumnOrColumnRange:any]) → string
```

Parameters

You can specify the following parameters:

- The optional parameter *Caption* of data type *string* designates the caption of the object.
- The optional parameter *StatisticsType* of data type *string* designates the statistics type of the object. Start entering the statistics type with a percentage sign, for example `%WorkingTime`.
- You can specify one of these statistics types:

Statistics type English	Statistics type German
%ResStates	%Stati
%MatFlowProperties	%Matflusseigenschaften
%RotationTime (*Turnplate*, *Turntable*, *PickAndPlace*)	%Drehungszeit (*Drehplatte*, *Drehtisch*, *PickAndPlace*)
%MovingTime (*Converter*)	%Umsetzzeit (*Umsetzer*)
%WorkingTime	%Arbeitszeit
%SetupTime	%Rüstzeit
%WaitingTime	%Wartezeit
%BlockedTime	%Blockiertzeit
%PowerUpDownTime	%HochHerunterfahrzeit
%StoppedTime	%Angehaltenzeit

Statistics type English	Statistics type German
%FailedTime	%Störungszeit
%PausedTime	%Pausenzeit
%EmptyTime	%Leerzeit
%MUMatFlowProperties	%BEMatflusseigenschaften
%MUEmptyTime	%BELeerzeit
%DrainCumulated	%SenkeKumuliert
%DrainAllTypes	%SenkeAlleTypen
%DrainTypesPortions (non summable)	%SenkeTypenanteile (nicht addierbar)
%DrainTypesTimes (non summable)	%SenkeTypenzeiten (nicht addierbar)
%Energy	%Energie
%MUClassStatistics	%BEKlassenStatistik
%MUClassTimePortions	%BEKlassenZeitanteile
%MUTimePortions	%BEZeitanteile
%TransUsageTime	%TransVerwendungszeit
%TransReadyTime	%TransBereitzeit
%TransPausedTime	%TransPausenzeit
%TransFailedTime	%TransStörungszeit
%TransTraveledDistance	%TransWegstrecke
%TransBatteryTime	%TransBatteriezeit
%Broker	%Broker
%BrokerMediationTime	%BrokerVermittlungsdauer
%BrokerDwellTime	%BrokerVerweildauer
%Exporter	%Exporter
%ExporterTimePortions	%ExporterZeitanteile
%WorkerTraveleledDistance	%WerkerWegstrecke
%ImporterWaiting	%ImporterWartend
%ImporterWaitingTime	%ImporterWartezeit
%ImporterMUWaitingTime	%ImporterBEWartezeit
%ImporterSetpWaitingTime	%ImporterRüstWartezeit

- The optional parameters *ColumnIndex*|*ColumnRange*|*ColumnName* of data type *integer* designate *ColumnIndex*|*ColumnRange*|*ColumnName* of the column. You have to start entering column names with the number of the respective column.

  Compare these examples for column identifiers:

Column identifier	Designates
#Sum	The column with the index or name **Sum**
0	The row index column, if available, otherwise it leads to an error
3	The column with the number 3
2..5	The columns 2 to 5
..5	All columns from the first available column to column 5
*..5	All columns from the first available column to column 5
5..	All columns from column 5 onward
5..*	All columns from column 5 onward
*	The last column of the table

  This syntax is the same as in the *HtmlReport*.

Return Value

The return value has the data type *string*.

Examples

```
print Source.getHtmlCode
```

```
Comment1.Cont := DataTable.getHtmlCode()            // entire table
Comment2.Cont := DataTable.getHtmlCode("caption")   // table with caption
Comment3.Cont := DataTable.getHtmlCode(1, 3)        // numerical column
index
Comment4.Cont := DataTable.getHtmlCode("1", 3)      // mixture of numerical/
string column indexes
Comment5.Cont := DataTable.getHtmlCode("1..2")      // column index range
Comment6.Cont := DataTable.getHtmlCode("#A")        // named column
```

See also

Display a HtmlReport

Display a Frame

getObservers [SimTalk]

Returns all observers of the object designated by <Path>.

Type

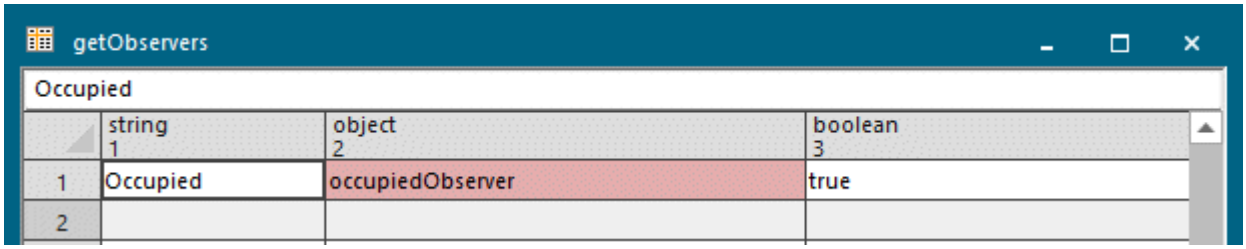
Method

Syntax

```
<Path>.getObservers -> table
```

Return Value

The return value has the data type *table*.



This table has three columns.

- Column 1 is of data type *string* and contains the name of the **Observed Value**.
- Column 2 is of data type *object* and contains the name of the **Method**.

  The **Method** in column 2 can contain a relative path to a *Method* or an *object reference*. If it is a relative path, you have to use the method *asString* of the *DataTable* to access the path in column 2.
- Column 3 is of data type *boolean* and contains true if the observer was created in the current object (**Created Here**), i.e., it is not inherited.

Example

```
// deletes all observers of a Station
var observerTable := Station.getObservers
for var row := 1 to observerTable.ydim
   if observerTable[3,row]             // only delete observers which are
not inherited
       if observerTable[2,row] /= void // observer is specified as an
object reference
           Station.removeObserver(observerTable[1,row],
observerTable[2,row])
       else                            // observer is specified by a
```

```
relative path
           Station.removeObserver(observerTable[1,row],
observerTable.asString(2,row))
        end
   end
next
```

See also

Edit Observers [general description]

New [observer]

asString [SimTalk] - DataTable

getXYWH [SimTalk]

Returns the coordinates of the position of the dialog of the object designated by <Path>, and assigns them to the local variables, which you specify.

Remarks

For a maximized window *getXYWH* returns -1 for the x-coordinate and the y-coordinate, and the correct size of the maximized window.

Type

Method

Syntax

```
<Path>.getXYWH(byRef X:integer, byRef Y:integer, byRef Width:integer, byRef
Height:integer)
```

Local Variables

You can specify the following local variables:

- The local variable *X* of data type *integer* designates the x-coordinate of the dialog of the object.
- The local variable *Y* of data type *integer* designates the y-coordinate.
- The local variable *Width* of data type *integer* designates its width.
- The local variable *Height* of data type *integer* designates its height.

For objects, which do not save their position and size, the *method* assigns -1, when the dialog is closed.

Example

```
Frame1.getXYWH(x,y,w,h)
print x," ",y," ",w," ",h
```

See also

setXYWH [SimTalk]

isNameUnique [SimTalk] - identifier

Returns if the *identifier* within the name space of the object designated by <Path> is unique (*true*), i.e., is not assigned yet, or if it is not unique (*false*), i.e., is already assigned.

Type

Method

Syntax

```
<Path>.isNameUnique(NameToBeChecked:string) → boolean
```

Parameter

The parameter *NameToBeChecked* of data type *string* designates the *name that is to be checked*.

Return Value

The return value has the data type *boolean*.

true if the name is unique.

false if the name to be checked is already assigned for the object designated by <Path>.

This object then either has a built-in attribute or a built-in method or a user-defined attribute with this identifier. When the object designated by <Path> is a *Frame* or a *folder*, then *isNameUnique* also considers objects with this identifier, which you inserted into the *Frame*.

Note

Use *isNameUnique* to find out in advance if the identifier is already assigned to an object within the surrounding *Frame* to prevent a runtime error if you want to rename an object with *SimTalk* commands.

Example

```
print .Hall2.isNameUnique("Station1")  // object inserted into Frame
print .Hall2.isNameUnique("Name")      // built-in attribute
print .Hall2.isNameUnique("~")         // built-in method
print .Hall2.isNameUnique("myUserDefinedAttribute")
```

See also

hasAttribute [SimTalk] - objects

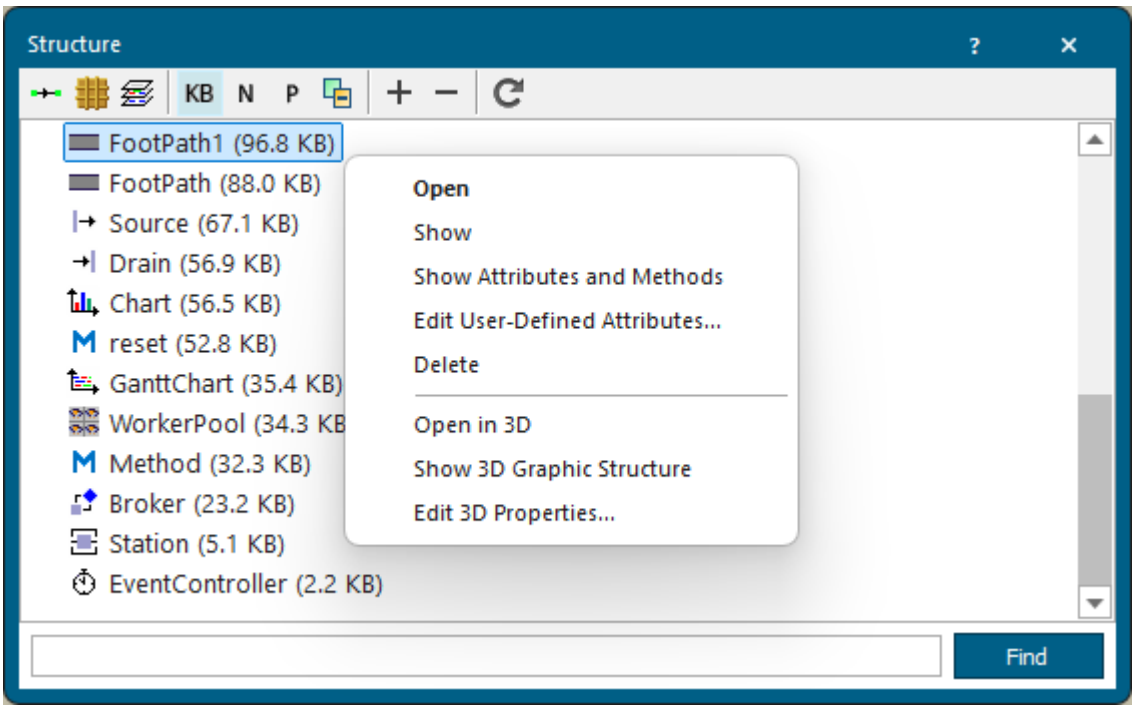
Namespace

memUsage [SimTalk]

Computes how much memory the object designated by <Path> uses.

Remarks

To show how much memory the individual objects in a *Frame* consume, you can also select the command **Show Memory Usage** on the context menu of the **Structure** window in the windows **Show Inheritance** and **Show Structure**.



Type

Method

Syntax

```
<Path>.memUsage → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print FootPath1.memUsage // might return 9685, i.e. 96.8 KB
print MyStation.memUsage -- might return 4975, i.e. 4.9 KB
```

See also

Show Structure [class library]

_3D.memUsage [SimTalk]

moveToFolder [SimTalk]

Moves the object designated by <Path> to the *folder* designated by the parameter *Folder*.

Type

Function

Syntax

```
<Path>.moveToFolder(Folder:Path) → object
```

Parameter

The parameter *Folder* of data type *path* designates the *folder* into which the object is to be moved.

Return Value

The return value has the data type *object*.

Example

```
.Resources.Exporter.moveToFolder(.Models)
```

See also

Move to Folder Control

MoveToFolderCtrl [SimTalk]

openDialog [SimTalk] - all objects

Opens the dialog box of the object designated by <Path>.

Remarks

openDialog also executes any **Open Control** you specified.

Type

Method

Syntax

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

Parameter

The optional parameter *CallOpenControl* of data type *boolean* determines if *Plant Simulation* opens the dialog box the same way as double-clicking the icon does (`true`) or not (`false`).

When you specify `false`, *Plant Simulation* just opens the dialog box, without executing any **Open Control**.

Default Value of the Parameter

The default value is false.

Return Value

The return value has the data type *boolean*.

true if *Plant Simulation* succeeded in opening the dialog box.

Example

```
Buffer.openDialog
.drill.openDialog(false)
basis.Materialflow.Source.openDialog(true)
self.~.~.&MyMethod.openDialog  // opens the window of the method
                               // MyMethod instead of executing it
```

See also

_3D.openDialog [SimTalk]

Open Control

removeAllObservers [SimTalk]

Deletes all observers of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.removeAllObservers → boolean
```

Return Value

The return value has the data type *boolean*.

true if inherited observers exist.

Example

```
MyStation.removeAllObservers
```

SimTalk

removeObserver [SimTalk]

addObserver [SimTalk]

See also

Delete [observer]

removeObserver [SimTalk]

Deletes the specified observer of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.removeObserver(AttributeName:string, Method:method/void) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the name of the **watchable** *attribute* or *read-only attribute*.
- The parameter *Method* of data type *method* designates the name of the method that is going to be executed.

  When you specify `void`, *Plant Simulation* deletes all observers which point to a non-existing *Method*.

Return Value

The return value has the data type *boolean*.

Example

```
MyStation.removeObserver("occupied", "myMethod")
MyStation.removeObserver("occupied", &myMethod)
```

SimTalk

removeAllObservers [SimTalk]

addObserver [SimTalk]

See also

Edit Observers [general description]

Delete [observer]

replace [SimTalk]

Replaces the object designated by *ObjectToReplace* with the object designated by <Path>.

Remarks

The object designated by <Path> has to be a class in the *Class Library*.

If the object designated by the parameter *ObjectToReplace* is a class, *Plant Simulation* merges both classes. This means that all instances of the class to be replaced will be replaced with instances of the replacing class and that the class to be replaced will be deleted.

If the object designated by the parameter *ObjectToReplace* is an instance, *Plant Simulation* replaces this instance with an instance of the replacing class.

Type

Method

Syntax

```
<Path>.replace(ObjectToReplace:object[,
RetainInheritedValues:boolean:=false])
```

Parameters

You can specify the following parameters:

- The parameter *ObjectToReplace* of data type *object* designates the object which you want to replace.
- The optional parameter *RetainInheritedValues* of data type *boolean* only takes effect if the object to be replaced is an instance. This means that the object to be replaced is located in a *Frame*. The parameter has no effect when merging classes.

  If the parameter *RetainInheritedValues* has the value `true`, *Plant Simulation* cuts inheritance for attributes of the instance to be replaced if the new origin class has another attribute value than the previous origin class.

  Default Value of the Parameter

  The default value is false.

Example

```
.MaterialFlow.Station.replace(.MaterialFlow.MyStation, false)
```

SimTalk

ReplacementMode [SimTalk]

replace [SimTalk]

See also

Replacing and Merging Objects with Drag-and-Drop

Load Object

Replacement Mode

setName [SimTalk]

Sets the name of the object designated by <Path> to the specified expression.

Remarks

You can type in a combination of letters, digits, and underscore (_) characters, for example `MyStation`, `MyStation1`, `My_Station_1`.

The name of an object cannot start with a digit, i.e., you cannot specify `1Station`.

> **Note**
> You cannot change the name of the *EventController* and of the *Connector*!

Type

Method

Syntax

```
<Path>.setName(NewName:string) → boolean
```

Parameter

The parameter *NewName* of data type *string* designates the name.

Return Value

The return value has the data type *boolean*.

As opposed to assigning the name to the attribute *Name*, the method *setName* returns if assigning the new name was successful or not.

Example

```
print MyStation.setName("MyStation") // returns true
```

See also

Name [general description]

Name [SimTalk] - general description

Rename [model or object]

setPosition [SimTalk]

Sets the position of the icon of the object designated by <Path> in the *Frame*.

Type

Method

Syntax

```
<Path>.setPosition(X:integer, Y:integer[,
CallMoveInFrameControl:boolean:=true])
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the x-coordinate of the object.
- The parameter *Y* of data type *integer* designates the y-coordinate.
- The optional parameter *CallMoveInFrameControl* of data type *boolean* determines if *Plant Simulation* also calls a **Move in Frame control** (`true`) or not (`false`).

  Default Value of the Parameter

  The default value is true.

  Specify `false` to not call the **Move in Frame control**.

Example

```
MyStation.setPosition(15,400)
ParallelStation.setPosition(80,200,true)
```

See also

Move in Frame Control

setXYWH [SimTalk]

Sets the position on the screen of the dialog box of the object designated by <Path>.

Remarks

setXYWH also applies for a *DataTable*, which you opened as a dialog in the foreground. Specifying a negative number for *X* or for *Y* centers the dialog on screen.

Type

Method

Syntax

```
<Path>.setXYWH(X:integer, Y:integer, Width:integer, Height:integer[,
TotalSize:boolean:=true])
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the x-coordinate of the dialog on screen.
- The parameter *Y* of data type *integer* designates the y-coordinate.
- The parameter *Width* of data type *integer* designates the width of the dialog.
- The parameter *Height* of data type *integer* designates the height of the dialog.

  Note

  The values **Width** and **Height** only apply for windows that you can resize, i. e., for **Object windows** of the **Frame**, the **Method**, the **Method Debugger**, the **DataQueue**, the **DataStack**, the **DataList**, the **DataTable**, and the **Icon Editor**.

- The optional parameter *TotalSize* of data type *boolean* sets if the *Width* designates the total width of the dialog box and the *Height* designates the total height of the dialog box (`true`) or the *Width* and the *Height* within the frame bordering the dialog box (`false`).

Examples

```
MyStation.setxywh(10,100,10,20)
Model1.setxywh(500,100,15,25)
```

```
MyDataTable.openDialogBox
MyDataTable.setxywh(-500,100,15,25)
```

See also

getXYWH [SimTalk]

showObject [SimTalk]

Selects the object designated by <Path> and shows it.

Remarks

For instances *Plant Simulation* selects the object designated by <Path> in the view of the *Frame* and shows it.

If the *Frame*, into which the object is inserted, is not open, *Plant Simulation* opens the *Frame* first.

For classes *Plant Simulation* expands the folder in the *Class Library* and selects the object.

Type

Method

Syntax

```
<Path>.showObject
```

Example

```
MyFrame.ParallelStation.showObject
.MaterialFlow.ParallelStation.showObject
```

updateDialog [SimTalk]

Updates the contents of the open dialog box of the object designated by <Path>.

Remarks

Updating the contents is sometimes necessary because *Plant Simulation* does not automatically show changed settings of an object, whose dialog box is open, for example by assignments in a method, in the dialog box itself.

Type

Method

Syntax

```
<Path>.updateDialog → boolean
```

Return Value

The return value has the data type *boolean*.

true if the dialog was updated succeeded in updating the dialog.

false if updating failed.

Example

```
EventController.StartDate := sysdate
EventController.updateDialog
```

See also

Refresh [on View menu]

writeObject [SimTalk]

Saves the object designated by <Path> to a file into the active folder (compare *setCurrentDirectory*) or into the folder whose absolute path you enter.

Remarks

writeObject applies to all classes and folders in the *Class Library*.

Type

Method

Syntax

```
<Path>.writeObject(FileName:string[, SaveAsLibrary:boolean:=false,
Password:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FileName* of data type *string* designates the name of the object file. You can, but you do not have to enter the file extension.
- The optional parameter *SaveAsLibrary* of data type *boolean* determines if *Plant Simulation* saves the designated **folder** as a library (true) or as a normal object (false).

  Default Value of the Parameter

  The default value is false.
- The optional parameter *Password* of data type *string* sets the password *Plant Simulation* uses when saving the designated file.

Example

```
.Informationflow.DataList.writeObject("MyDataList.psobj")
.Materialflow.ParallelStation.writeObject("mypp.psobj"
.ApplicationObjects.Productionsystem.writeObject("MyLibrary.pslib", true)
```

SimTalk

setLibraryInfo [SimTalk]

setCurrentDirectory [SimTalk]

See also

Save Object As

Save Folder As

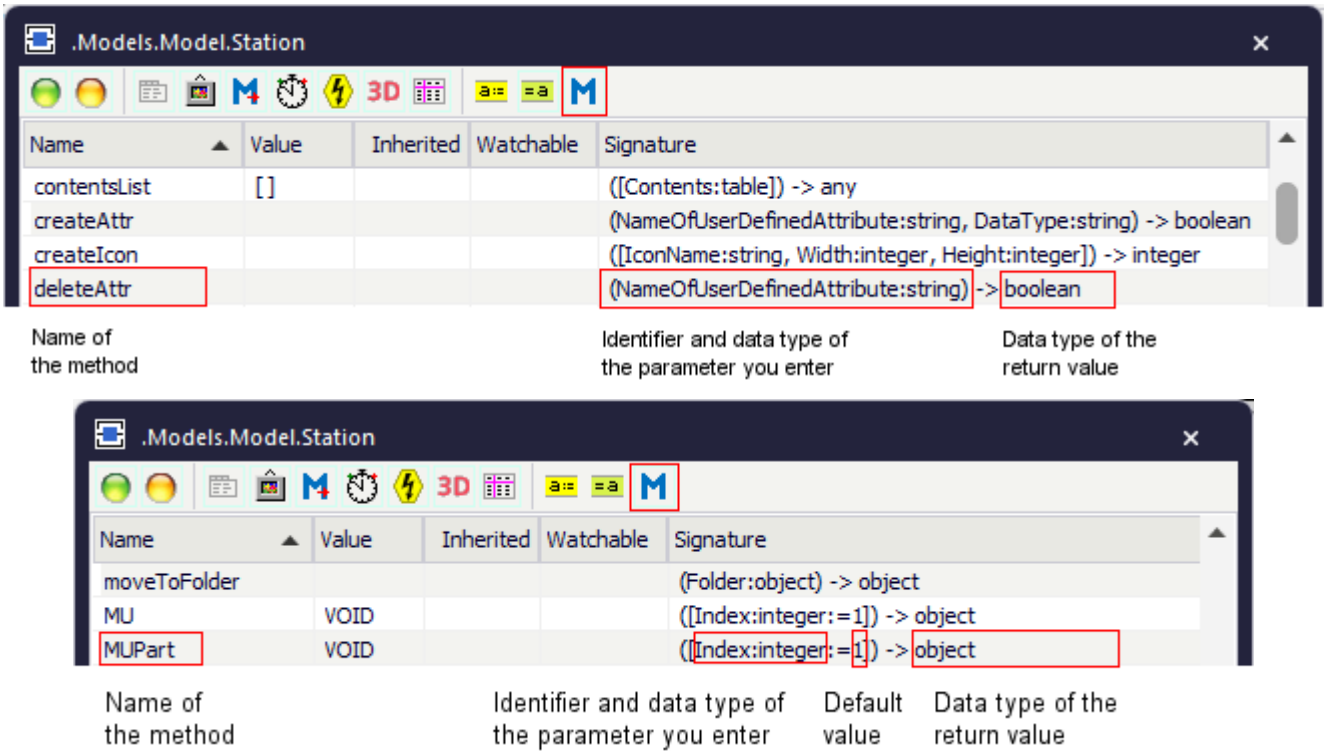
Save Library As

Methods for Object Icons

_Methods of Object Icons

The objects provide the methods listed in the table of contents to the left to **animate them**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected **Class [general description]**.

- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected **Instance [general description]**.

createIcon [SimTalk]

Creates a new icon for the object designated by <Path>.

Remarks

For the newly created icon the setting **Activate Transparency** is activated and the background is filled with the transparency color.

> **Note**
> The method applies to the icon of the addressed instance, not to icon of the object class.

Type

Method

Syntax

```
<Path>.createIcon(IconName:string, Width:integer, Height:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *IconName* of data type *string* designates the name of the icon.
- The parameter *Width* of data type *integer* designates the width of the icon.
- The parameter *Height* of data type *integer* designates its height.

Return Value

The return value has the data type *integer*.

Example

```
MyStation.createIcon("state",20,30)
```

SimTalk

deleteIcon [SimTalk]

See also

Activate Transparency

New Icon

Icon Editor

deleteIcon [SimTalk]

Deletes the icon of the object designated by <Path>.

Remarks

The method applies to the icon of the addressed instance, not to an icon of the object class.

Type

Method

Syntax

```
<Path>.deleteIcon(IconName:string)
<Path>.deleteIcon(IconNumber:integer)
```

Parameter

The parameter *IconName* of data type *string* designates the *name of the icon*.

Instead, you can also specify the *number of the icon* as the parameter *IconNumber* of data type *integer*.

Example

```
MyParallelStation.deleteIcon("MyIcon")
MyParallelStation.deleteIcon(1)
```

SimTalk

createIcon [SimTalk]

existsIcon [SimTalk]

Checks if the icon of the object designated by <Path> exists (*true*) or does not exist (*false*).

Type

Method

Syntax

```
<Path>.existsIcon(IconName:string) → boolean
<Path>.existsIcon(IconNNumber:integer) → boolean
```

Parameter

The parameter *IconName* of data type *string* designates the *name of the icon*.

Instead, you can also specify the *number of the icon* as the parameter *IconNumber* of data type *integer*.

Return Value

The return value has the data type *boolean*.

Example

```
press.existsIcon("failed")
press.existsIcon(2)
```

getIconSize [SimTalk]

Returns the width and the height of the current icon of the object designated by <Path>.

Remarks

Plant Simulation then assigns the returned width and the height to the local variables, whose names you specify.

Type

Method

Syntax

```
<Path>.getIconSize(byRef Width:integer, byRef Height:integer)
```

Local Variables

You can specify the following local variables:

- The local variable *Width* of data type *integer* designates the width of the current icon.
- The local variable *Height* of data type *integer* designates its height.

Example

```
var w,h: integer
MyStation.getIconSize(w,h)
print w," ",h
```

SimTalk

setIconSize [SimTalk]

See also

Set Icon Size

getPixel [SimTalk]

Returns the RGB value of a pixel denoted by the parameters of the object designated by <Path>.

Remarks

The *method* applies to all objects, except for the *Variable*, the *Comment*, the *folder* and the *toolbar*.

Type

Method

Syntax

```
<Path>.getPixel(X:integer, Y:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the x-coordinate of the pixel.
- The parameter *Y* of data type *integer* designates its y-coordinate.

Return Value

The return value has the data type *integer*.

Example

```
var rgb, red, green, blue: integer
rgb := MyStation.getPixel(1,1)
red := rgb mod 256
green := (rgb div 256) mod 256
blue := rgb div (256*256)
```

SimTalk

setPixel [SimTalk]

See also

Pick Color

Replace Color

putIconToClipboard [SimTalk]

Opens the Windows clipboard and stores the current icon of the object designated by <Path> in CF_BITMAP format.

Remarks

putIconToClipboard may fail because:

- The Clipboard is locked by another application.
- The icon designated by the parameter does not exist.

Type

Method

Syntax

```
<Path>.putIconToClipboard(IconName:string) → boolean
<Path>.putIconToClipboard(IconNumber:integer) → boolean
```

Parameter

The parameter *IconName* of data type *string* designates the *name of the icon*.

Instead, you can also enter the *number of the icon* as the parameter *IconNumber* of data type *integer*.

Return Value

The return value has the data type *boolean*.

true if the assignment was successful.

false if the assignment failed.

Example

```
MyStation.putIconToClipboard(3)
```

See also

setCurrIconFromClipboard [SimTalk]

saveIconToFile [SimTalk]

Saves the icon of the object designed by <Path> as a png file to the *Plant Simulation* installation folder.

Type

Method

Syntax

```
<Path>.saveIconToFile(IconName:string, FileName:string) → boolean
<Path>.saveIconToFile(IconNumber:string, FileName:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *IconName* of data type *string* designates the *name of the icon*.

  Instead, you can also enter the *number of the icon* as the parameter *IconNumber* of data type *integer*.
- The parameter *FileName* of data type *string* designates the filename with which *Plant Simulation* saves the graphics file.

Return Value

The return value has the data type *boolean*.

false if saving the icon to a file failed.

Example

```
sp1.saveIconToFile(3,"iconfile")
sp1.saveIconToFile("icon_name","iconfile")
```

See also

Export Icon to File

setCurrIconFromClipboard [SimTalk]

Opens the clipboard, reads the bitmap stored there, and converts it to the *Plant Simulation* format.

Remarks

After conversion *Plant Simulation* assigns the bitmap to the icon defined by the parameter *IconName* of data type *string*, or the number defined by the parameter *IconNumber* of data type *integer* and sets it as the current icon of the object designed by <Path>.

> **Note**
> The method applies to the icon of the addressed instance, not to the icon of the object class.

setCurrIconFromClipboard may fail because:

- The Clipboard is locked by another application.
- The Clipboard does not contain an image in CF_BITMAP format.
- The icon designated by the parameter does not exist.

Type

Method

Syntax

```
<Path>.setCurrIconFromClipboard(IconName:string) → boolean
<Path>.setCurrIconFromClipboard(IconNumber:integer) → boolean
```

Parameter

The parameter *IconName* of data type *string* designates the *name of the icon*.

Instead, you can also specify the *number of the icon* as the parameter *IconNumber* of data type *integer*.

Return Value

The return value has the data type *boolean*.

true if the assignment was successful.

false if the assignment failed.

Example

```
MyFrame.setCurrIconFromClipboard(3)
MyFrame.setCurrIconFromClipboard("MyIcon")
```

See also

Paste Contents of the Clipboard [Home ribbon]

setIconFromFile [SimTalk]

Sets the icon of the object designed by <Path> to a number and overwrites the icon of the file with a graphics file.

Remarks

setIconFromFile applies to the icon of the addressed instance, not to icon of the object class.

Type

Method

Syntax

```
<Path>.setIconFromFile(IconName:string, FileName:string) → boolean
<Path>.setIconFromFile(IconNumber:integer, FileName:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *IconName* of data type *string* designates the *name of the icon*.

  Instead, you can also enter the *number of the icon* as the parameter *IconNumber* of data type *integer*.
- The parameter *FileName* of data type *string* designates the file name of the graphic, which overwrites the current icon.

  The graphics file of the icon may be located in a folder of your choice.

Return Value

The return value has the data type *boolean*.

false if the method fails and does not make changes.

Example

```
MyStation.setIconFromFile(3,"outOfOrder.gif")
MyStation.setIconFromFile("failed","C:\graphics\outOfOrder.gif")
```

See also

Import

setIconSize [SimTalk]

Sets the size, i.e., the width and the height of the current icon of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.setIconSize(Width:integer, Height:integer)
```

Parameters

You can specify the following parameters:

- The parameter *Width* of data type *integer* designates the width of the current icon.
- The parameter *Height* of data type *integer* designates its height.

Example

```
MyStation.setIconSize(44, 44)
```

SimTalk

getIconSize [SimTalk]

Set Icon Size

setPixel [SimTalk]

Sets the pixel of the point of the icon of the object designated by <Path>.

Remarks

Since all objects of the same class share their icons, *Plant Simulation* changes the pixel in the class and in all derivatives.

> **Note**
> The *method* applies to all objects, except for the *Variable*, the *Comment*, the *folder*, and to the *toolbar*.

Type

Method

Syntax

```
<Path>.setPixel(X:integer, Y:integer, RGB:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the x-coordinate of the pixel of the icon designated by <Path>.
- The parameter *Y* of data type *integer* designates its y-coordinate.
- The parameter *RGB* of data type *integer* designates its color.
- Set the RGB values of the color with the method *makeRGBValue*.

Return Value

The return value has the data type *integer*.

Example

```
// sets the pixel in the upper left corner to pink
MyStation.setPixel(1,1,makeRGBValue(255,0,255))
MyStation.redraw
```

SimTalk

getPixel [SimTalk]

makeRGBValue [SimTalk]

See also

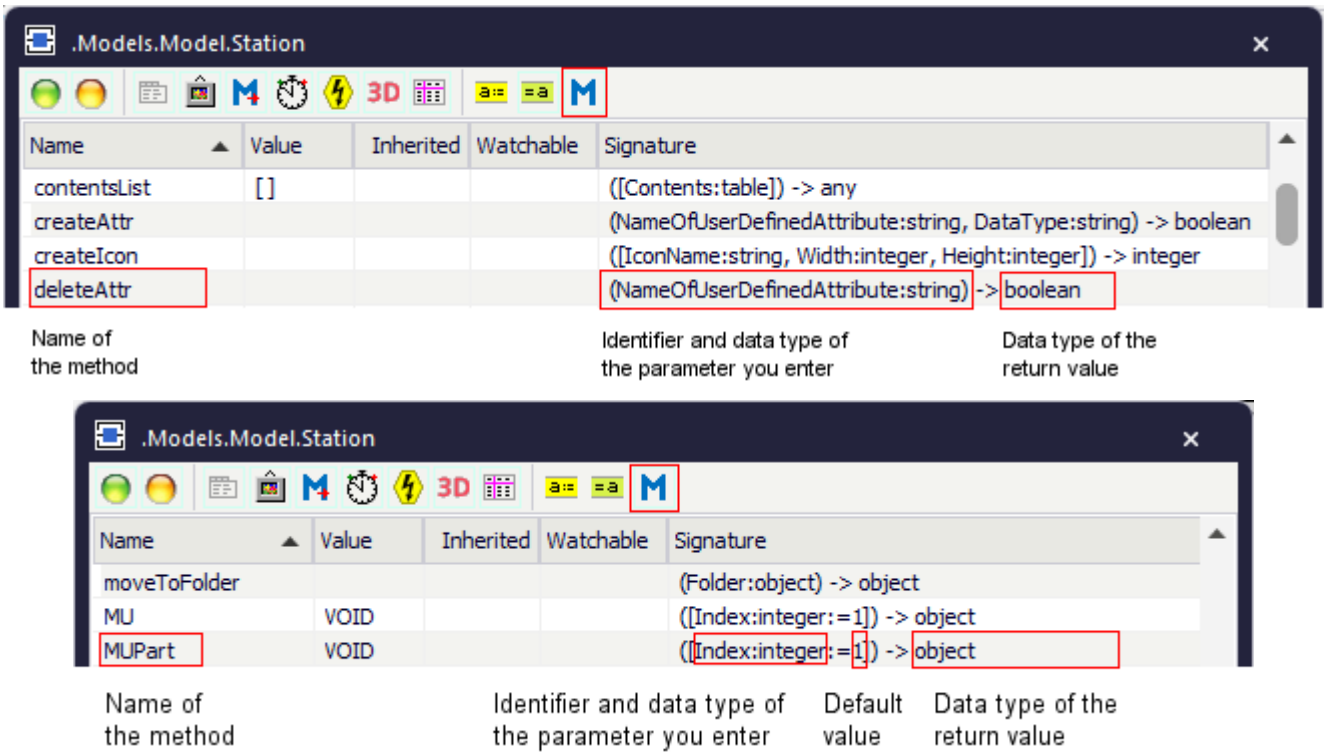
Replace Color

Methods for the Location, Predecessor, and Successor

_Methods for the Location, Predecessor, and Successor

Most of the objects provide the *methods* listed in the table of contents to the left for querying their location, their predecessor, and their successor.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected **Class [general description]**.

- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected **Instance [general description]**.

pred [SimTalk] - general description

Returns the direct predecessor of the object designated by <Path>. Or returns the predecessor with the specified number of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.pred → object
<Path>.pred([PredecessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *PredecessorNumber* of data type *integer* designates the number of the predecessor.

Default Value of the Parameter

The default value is 1, meaning the first predecessor.

Return Value

The return value has the data type *object*.

> **Note**
>
> If the specified predecessor of the object is an *Interface*, the *method* does not return the *Interface* itself, but the predecessor of the *Interface*. If this successor in turn is another *Interface*, *Plant Simulation* follows it until it reaches a material flow object, which can receive *MUs*.
>
> If the specified predecessor of the object is a *Frame*, *Plant Simulation* does not return the *Frame* itself, but the predecessor of the *Interface* inserted into this *Frame*.

Example

```
if model.pred(3).name = "press"
...
end
```

SimTalk

pred [SimTalk] - lane A or B

predConnector [SimTalk] - general description

NumPred [SimTalk] - general description

predConnector [SimTalk] - general description

Returns the connection to the immediate predecessor of the object designated by <Path>. Or returns the connection to the predecessor designated by its number of the object designated by <Path>.

Remarks

Adding and deleting connections may change the index number of a connection.

Type

Method

Syntax

```
<Path>.predConnector → object
<Path>.predConnector([PredecessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *PredecessorNumber* of data type *integer* designates the number of the predecessor.

Default Value of the Parameter

The default value is 1, meaning the first predecessor.

Return Value

The return value has the data type *object*.

VOID if the connection does not exist.

Example

```
for var i := MyTrack.NumPred downto 1 // deletes connections with
predecessors
   MyTrack.predConnector(i).deleteObject
next
```

SimTalk

pred [SimTalk] - general description

predConnector [SimTalk] - lane A or B

succ [SimTalk] - general description

Returns the direct successor of the object designated by <Path>. Or returns the successor with the specified number of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.succ → object
<Path>.succ([SuccessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *SuccessorNumber* of data type *integer* designates the number of the successor.

Default Value of the Parameter

The default value is 1, meaning the first successor.

Return Value

The return value has the data type *object*.

Note

If the specified successor of the object is an *Interface*, the *method* does not return the *Interface* itself, but the successor of the *Interface*. If this successor in turn is another *Interface*, *Plant Simulation* follows it until it reaches a material flow object, which can receive *MUs*.

If the specified successor of the object is a *Frame*, *Plant Simulation* does not return the *Frame* itself, but the successor of the *Interface* inserted into this *Frame*.

If an *Interface* has a single successor, this successor is uniquely identified and *Plant Simulation* uses it independently of the **Exit Strategy** of the *Interface*.

If an *Interface* has several successors, *Plant Simulation* determines the successor using its **Exit Strategy**. If you selected a blocking **Exit Strategy**, *Plant Simulation* determines the successor no matter if this successor can receive a *MU* at the moment or not. If you selected a non-blocking **Exit Strategy**, *Plant Simulation* tries to determine the next successor, which can receive *MUs*, according to the selected **Exit Strategy**. If no successor can receive a *MU*, the *method* returns VOID. The successor, which will be able to receive *MUs* next cannot be determined at this point in time. This does not apply if there is only one successor.

Example

```
@.move(ParallelStation.succ(3))
```

SimTalk

succ [SimTalk] - lane A or B

succConnector [SimTalk] - general description

NumSucc [SimTalk] - general description

succConnector [SimTalk] - general description

Returns the connection to the immediate successor of the object designated by <Path>. Or returns the connection to the successor with the designated number of the object designated by <Path>.

Remarks

Adding and deleting connections may change the index number of a connection.

Type

Method

Syntax

```
<Path>.succConnector → object
<Path>.succConnector([SuccessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *SuccessorNumber* of data type *integer* designates the number of the successor.

Default Value of the Parameter

The default value is 1, meaning the first successor.

Return Value

The return value has the data type *object*.

VOID if the *Connector* does not exist.

Example

```
MyTrack.succConnector.deleteObject
// delete the connector to the succeeding object
```

SimTalk

succ [SimTalk] - general description

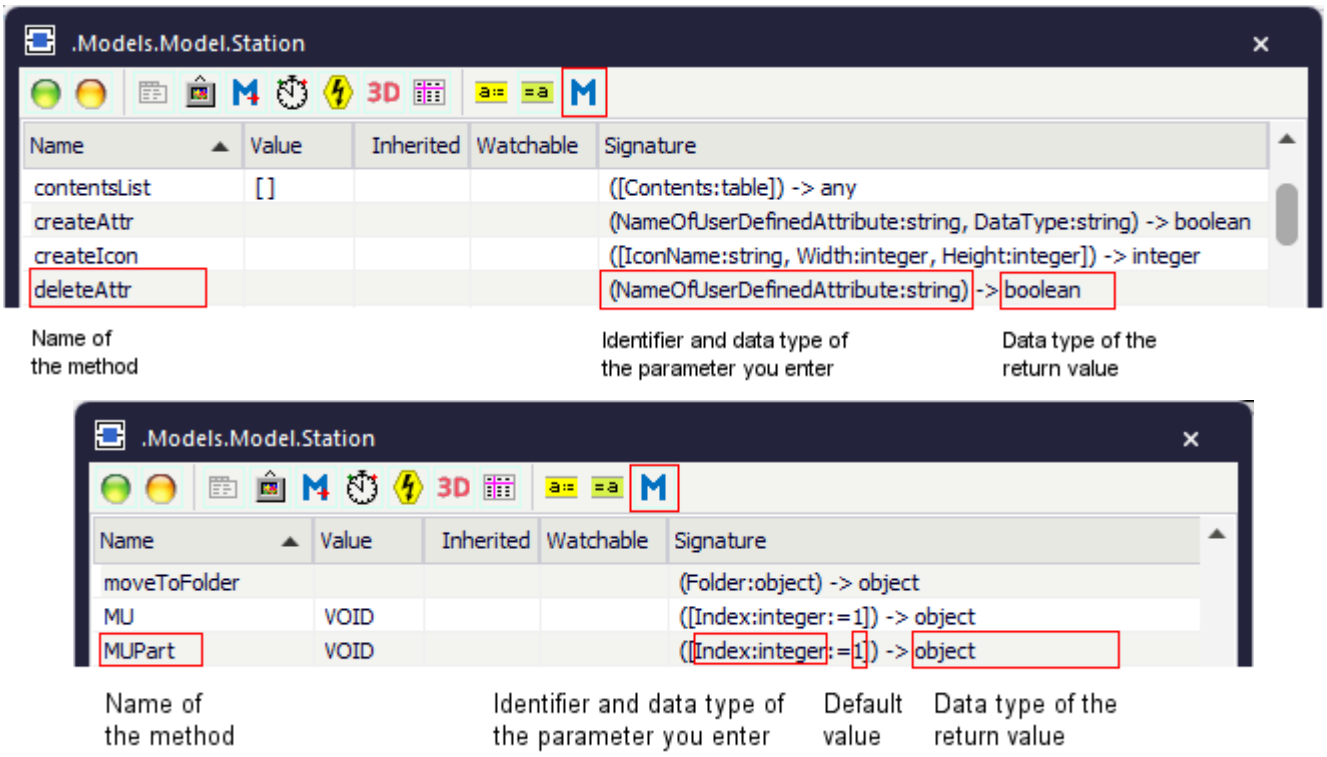
succConnector [SimTalk] - lane A or B

Methods for Managing Inheritance Relations

_Methods for Managing Inheritance Relations

Most objects provide the *methods* listed in the table of contents to the left for returning inheritance relations.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected **Instance [general description]**.

childNo [SimTalk]

Returns all instances of the class of the object designated by <Path>.

Remarks

This is independent of the name of the instance or the *Frame* into which the instance is inserted.

Type

Method

Syntax

```
<Path>.childNo(Number:integer) → object
```

Parameter

The parameter *Number* of data type *integer* designates the number in the list of children (*instances*) of the object. You can query the size of the list with the read-only attribute *NumChildren*.

Return Value

The return value has the data type *object*.

Example

```
for var index := 1 to .MUs.Transporter.NumChildren
   print .MUs.Transporter.childNo(index)
   // outputs all children
next
```

SimTalk

NumChildren [SimTalk]

See also

Instance [general description]

hasAttribute [SimTalk] - objects

Returns if the object designated by <Path> has an attribute or a method with the designated name (true) or not (false).

Type

Method

Syntax

```
<Path>.hasAttribute(AttributeName:string) → boolean
```

Parameter

The parameter *AttributeName* of data type *string* designates the *name of the attribute* of which you want to know if the object has it or not.

Return Value

The return value has the data type *boolean*.

true if the object has the attribute that can be read.

false for the parameter `Imp`, `SetupImp`, and `FailImp` if this attribute does not exist for the specified object.

Example

```
print Station.hasAttribute("Availability")
```

```
print Conveyor.hasAttribute("Imp")       -- returns false
print Conveyor.hasAttribute("FailImp")   -- returns false
print Conveyor.hasAttribute("SetupImp")  -- returns false
```

SimTalk

_3D.hasAttribute [SimTalk]

isNameUnique [SimTalk] - identifier

inheritAttribute [SimTalk] - object

Activates inheritance of the specified attribute of the object designated by <Path> if it was deactivated.

Type

Method

Syntax

```
<Path>.inheritAttribute(AttributeName:string)
```

Parameter

The parameter *AttributeName* of data type *string* designates the name of the attribute.

Example

```
Conveyor.inheritAttribute("Length")
Conveyor.Sensors.ID1.inheritAttribute("Position")
Station.Failures.Failure.inheritAttribute("Availability")
Station.Imp.inheritAttribute("Priority")
Station._3D.inheritAttribute("VisibleGraphicGroups")
&Variable.inheritAttribute("Value")
```

SimTalk

_3D.inheritAttribute [SimTalk]

See also

Instance

typeOf [SimTalk]

Returns if the object designated by <Path> has the same internal class type as the comparison object, i.e., if both have the same internal class type (true) or not (false).

Type

Method

Syntax

```
<Path>.typeOf(ObjectToBeCompared:object) → boolean
```

Parameter

The parameter *ObjectToBeCompared* of data type *object* designates the comparison object.

Return Value

The return value has the data type *boolean*.

Example

```
print MyStation.typeOf(station2)
// returns true if each station is of type Station
```

See also

inheritAttribute [SimTalk] - object

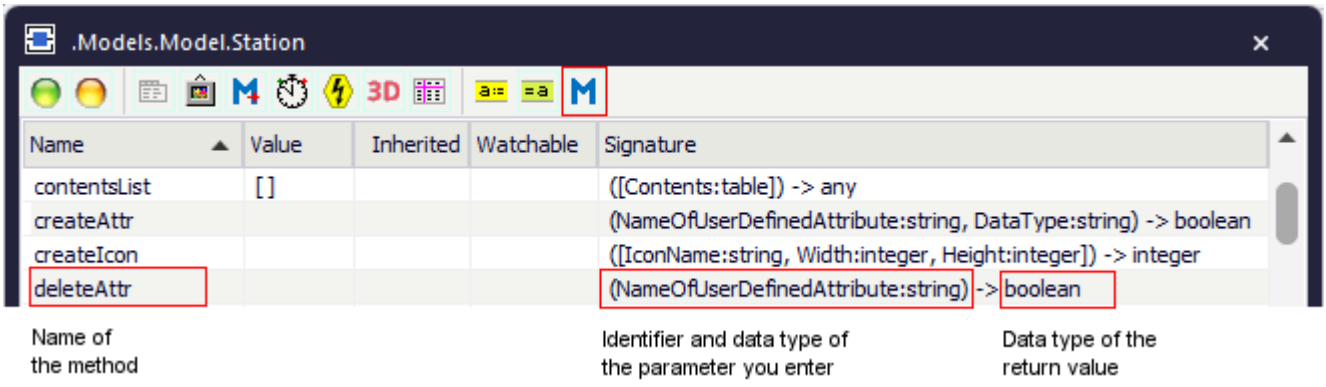
_3D.InternalClassType [SimTalk]

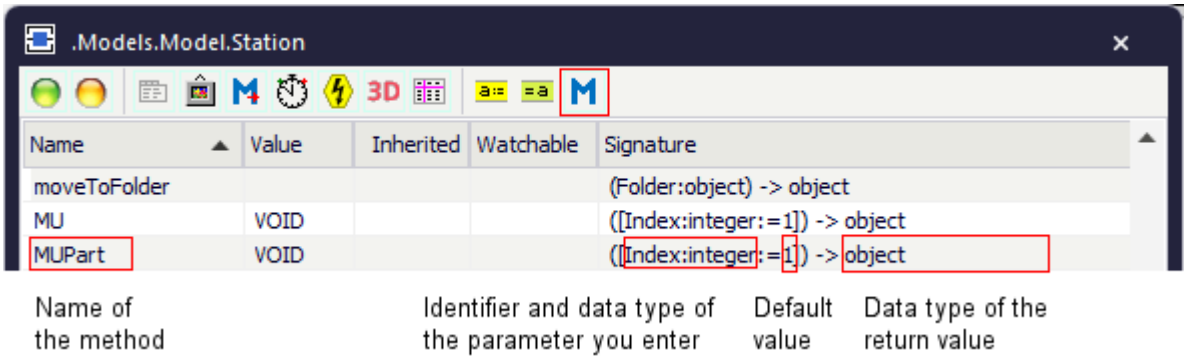
Methods for Managing Attributes

_Methods for Managing Attributes

Most objects provide the *methods* listed in the table of contents to the left for managing their attributes.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.





You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected **Instance [general description]**.

getAttribute [SimTalk] - object

Returns the value of an attribute of the object designated by <Path>.

Remarks

If the attribute is a *user-defined attribute* of data type *method*, *getAttribute* returns the method itself.

```
var bInherited:boolean
Station.getAttribute("MyMethodAttribute", bInherited)
// does not call MyMethodAttribute
```

To get the result of the method call instead of the method itself, you have to explicitly call the method:

```
Station.getAttribute("MyMethodAttribute").execute
```

Type

Method

Syntax

```
<Path>.getAttribute(AttributeName:string[, byRef Inherited:boolean, byRef
CanInherit:boolean]) → any
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the value of the attribute.
- When a local variable of data type *boolean* is passed to the optional parameter *Inherited*, *Plant Simulation* assigns **true** to this variable if the attribute designated by string *inherits* its value, **false** if it does not inherit the value.

  In case an attribute cannot be inherited, the optional parameter *Inherited* always returns false.
- When a local variable of data type *boolean* is passed to the optional parameter *CanInherit*, *Plant Simulation* assigns **true** to this variable if the value of the attribute designated by *string* can be inherited, **false** if it cannot be inherited.

Return Value

The return value has the data type *any*.

Examples

```
print MyStation.getAttribute("ProcTime")
```

```
// checks if inheritance of the attribute 'Accumulating' is active or not
var inherited: boolean
print MyConveyor.getAttribute("Accumulating", inherited)
print inherited
```

```
var ResultOfMethod :=
Station.getAttribute("MethodAttributeWithoutParameters")
var Result :=
Station.getAttribute("MethodAttributeWith2Parameters").execute(5, 9)
```

```
// Check if the source code of the Method is inherited
var bInherit:boolean

&Method.getAttribute("Program", bInherit)
print bInherit
```

SimTalk

getSubAttribute [SimTalk]

setSubAttribute [SimTalk]

setAttribute [SimTalk] - object

putAttributeNamesIntoTable [SimTalk] - objects

See also

Inheritance [general description]

Inherit Source Code

isNameUnique [SimTalk] - identifier

_3D.getAttribute [SimTalk]

getSubAttribute [SimTalk]

Returns the value of a sub-attribute of the specified attribute of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.getSubAttribute(AttributeName:string, SubAttributeName:string) → any
<Path>.getSubAttribute(AttributeName:string, SubAttributeName:string[,
Inherited:boolean]) → any
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the attribute.
- The parameter *SubAttributeName* of data type *string* designates the value of the sub-attribute.
- The optional parameter *Inherited* of data type *boolean* determines if the method ignores the value, which you enter as the parameter of data type *boolean*, and returns, if the attribute is inherited (true) or not (false).

Return Value

The return value has the data type *any*.

Examples

```
var b: boolean
print MyStation.getSubAttribute("ProcTime","Type", b)
print b // the return value true means that the attribute is inherited,
        // the return value false that it is not inherited
```

```
print MyStation.getSubAttribute("ProcTime","Type") // "Normal"
print MyStation.getSubAttribute("ProcTime","Mu")   // 1:00.0000
```

SimTalk

getAttribute [SimTalk] - object

putAttributeNamesIntoTable [SimTalk] - objects

setAttribute [SimTalk] - object

setTypeAndAttr [SimTalk]

setSubAttribute [SimTalk]

putAttributeNamesIntoTable [SimTalk] - objects

Writes the names and the data types of all attributes of the object designated by <Path> into a table.

Remarks

If you do not specify any optional parameters, the method does not return the *read-only attributes*.

Type

Method

Syntax

```
<Path>.putAttributeNamesIntoTable(Table:table)
<Path>.putAttributeNamesIntoTable(Table:table[,
IncludeUserDefinedAttribute:boolean:=false, IncludeMethods:boolean:=false,
Language:integer:=ModelLanguage])
```

Parameters

You can specify the following parameters:

- The parameter *Table* of data type *table* designates the table.

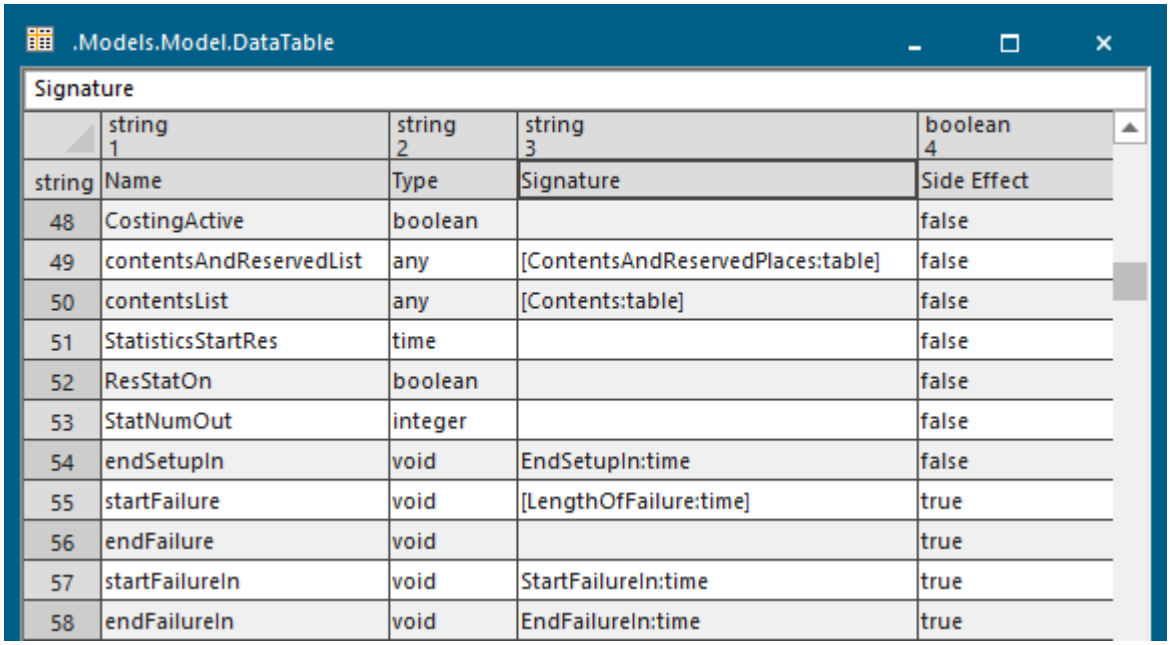
  The column named **Signature** of the parameter *Table* contains the complete signature of a *built-in Method*, instead of only the parameters of this *built-in Method*.

  The column named **Side Effect** contains a boolean value, which designates if read access has a side effect or only returns a value.

  Example

  ```
  Station.putAttributeNamesIntoTable(DataTable,true,true,UserInterfaceLangua
  ge)
  ```

  The instructions above return this *DataTable*:



.Models.Model.DataTable

Signature

	string 1	string 2	string 3	boolean 4
string	Name	Type	Signature	Side Effect
48	CostingActive	boolean		false
49	contentsAndReservedList	any	[ContentsAndReservedPlaces:table]	false
50	contentsList	any	[Contents:table]	false
51	StatisticsStartRes	time		false
52	ResStatOn	boolean		false
53	StatNumOut	integer		false
54	endSetupIn	void	EndSetupIn:time	false
55	startFailure	void	[LengthOfFailure:time]	true
56	endFailure	void		true
57	startFailureIn	void	StartFailureIn:time	true
58	endFailureIn	void	EndFailureIn:time	true

- The optional parameter *IncludeUserDefinedAttribute* of data type *boolean* determines if *Plant Simulation* also writes user-defined attributes into the table (`true`). If you enter `false`, *Plant Simulation* only writes the names of the built-in attributes into the table.

  Default Value of the Parameter

  The default value is false.

- The optional parameter *IncludeMethods* of data type *boolean* determines if *Plant Simulation* not only writes the names of the attributes into the table, but also those of the methods and of the read-only attributes (`true`). If the parameter *IncludeUserDefinedAttribute* is `true` as well, *Plant Simulation* also writes the names of the user-defined methods and read-only attributes into the table.

  Default Value of the Parameter

  The default value is false.

- The optional parameter *Language* of data type *integer* sets the language in which *Plant Simulation* shows the names of the attributes. `0` designates **German**, `1` designates **English**. If you do not specify this parameter, *Plant Simulation* shows the names of the attributes in the language of the model.

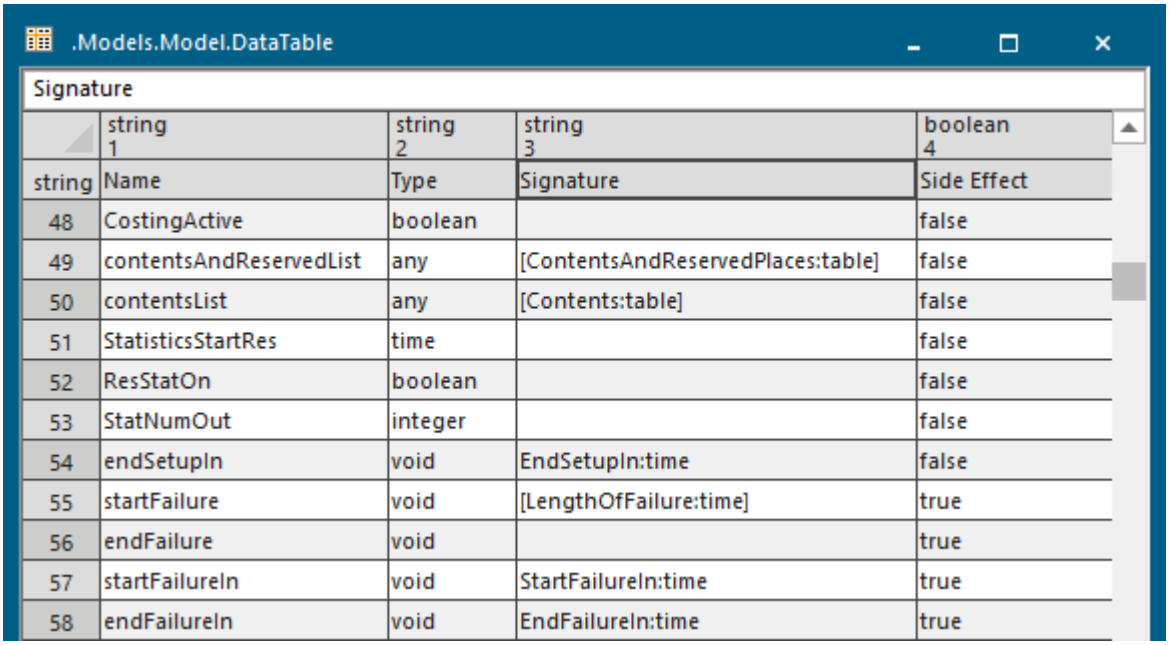
  Default Value of the Parameter

  The default value is **ModelLanguage**.

Example

```
MyStation.putAttributeNamesIntoTable(MyDataTable)
```

The instructions above return this *DataTable*:


.Models.Model.DataTable

Signature

	string 1	string 2	string 3	boolean 4
string	Name	Type	Signature	Side Effect
48	CostingActive	boolean		false
49	contentsAndReservedList	any	[ContentsAndReservedPlaces:table]	false
50	contentsList	any	[Contents:table]	false
51	StatisticsStartRes	time		false
52	ResStatOn	boolean		false
53	StatNumOut	integer		false
54	endSetupIn	void	EndSetupIn:time	false
55	startFailure	void	[LengthOfFailure:time]	true
56	endFailure	void		true
57	startFailureIn	void	StartFailureIn:time	true
58	endFailureIn	void	EndFailureIn:time	true

SimTalk

getAttribute [SimTalk] - object	language [SimTalk]
getSubAttribute [SimTalk]	userInterfaceLanguage [SimTalk]
setAttribute [SimTalk] - object	isNameUnique [SimTalk] - identifier
setSubAttribute [SimTalk]	

setAttribute [SimTalk] - object

Sets the value of the specified attribute of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.setAttribute(AttributeName:string, Value:any)
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the attribute.
- The parameter *Value* of data type *any* designates the value.

Example

```
MyStation.setAttribute("ProcTime",5)
```

SimTalk

getAttribute [SimTalk] - object

getSubAttribute [SimTalk]

putAttributeNamesIntoTable [SimTalk] - objects

setSubAttribute [SimTalk]

See also

isNameUnique [SimTalk] - identifier

setSubAttribute [SimTalk]

Sets the value of a sub-attribute of the specified attribute of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.setSubAttribute(AttributeName:string, SubAttributeName:string,
Value:any)
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the attribute.
- The parameter *SubAttributeName* of data type *string* designates the sub-attribute.
- The parameter *Value* of data type *any* designates the value of the sub-attribute.

Example

```
MyStation.setSubAttribute("ProcTime","Type","uniform")
```

SimTalk

getAttribute [SimTalk] - object

getSubAttribute [SimTalk]

putAttributeNamesIntoTable [SimTalk] - objects

setAttribute [SimTalk] - object

See also

isNameUnique [SimTalk] - identifier

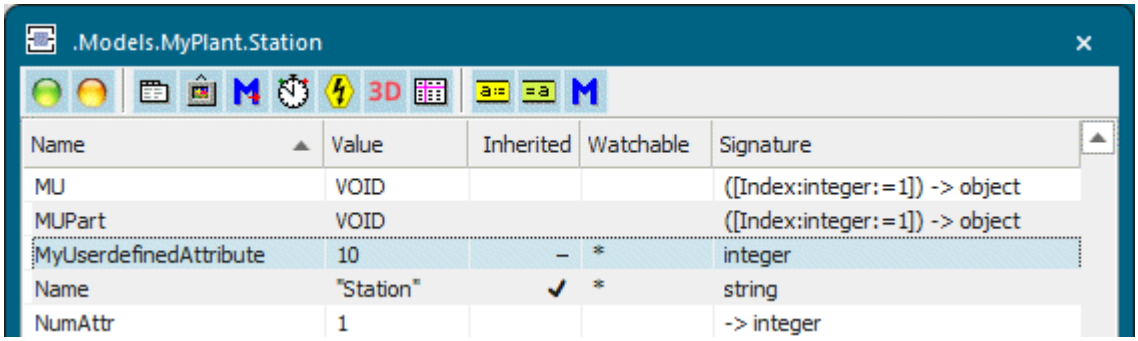
setTypeAndAttr [SimTalk]

Methods for Managing User-defined Attributes

_Methods of User-defined Attributes

Most objects provide the *methods* listed in the table of contents to the left for working with the *user-defined attributes* of the objects, which you define on the **Tab User-defined**.

The window **Show Attributes and Methods** shows the user-defined attribute itself and its value.



See also

User-defined Attributes [general description]

Create a User-defined Attribute Manually

Create a User-defined Attribute During the Simulation

_Attributes of User-defined Attributes

createAttr [SimTalk]

Creates the designated *user-defined attribute* for the object designated by <Path>.

Remarks

The attribute will be propagated to all of its instances.

Type

Method

Syntax

```
<Path>.createAttr(AttributeName:string, DataType:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the name of the *user-defined attribute*.
- The parameter *DataType* of data type *string* designates its data type.

Return Value

The return value has the data type *boolean*.

false if the new name or the data type is not valid.

Example

```
MyStation.createAttr("lotsize","integer")
MyStation.createAttr("inventoryNo","string")
@.createAttr("Paint","boolean")
```

SimTalk

deleteAttr [SimTalk]

See also

User-defined Attributes [general description]

Create a User-defined Attribute During the Simulation

New [user-defined attribute]

deleteAttr [SimTalk]

Deletes the designated *user-defined attribute* of the object designated by <Path>.

Remarks

You can only delete *user-defined attributes* for the object for which you created them and for which they are not inherited. *Plant Simulation* deletes the *user-defined attributes* from all instances of the object.

> **Note**
> Deleting an object will automatically also delete all of its *user-defined attributes*.

Type

Method

Syntax

```
<Path>.deleteAttr(AttributeName:string) → boolean
```

Parameter

The parameter *AttributeName* of data type *string* designates the *user-defined attribute*.

Return Value

The return value has the data type *boolean*.

Examples

```
.MUs.Part.deleteAttr("lotsize")
```

```
@.deleteAttr("Paint")
```

SimTalk

createAttr [SimTalk]

See also

User-defined Attributes [general description]

Create a User-defined Attribute During the Simulation

Delete [user-defined attribute]

getAttribute [SimTalk] - user-defined attribute

Returns the inheritance status of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.<UserDefinedAttribute>.getAttribute(AttributeName:string, byref
Inherited:boolean, byref CanInherit:boolean) -> any
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the name of the attribute of the *user-defined attribute*.
- If a local variable of data type *boolean* is passed to the parameter *Inherited*, *Plant Simulation* assigns true to this variable if the attribute designated by string *inherits* its value, false if it does not inherit the value.

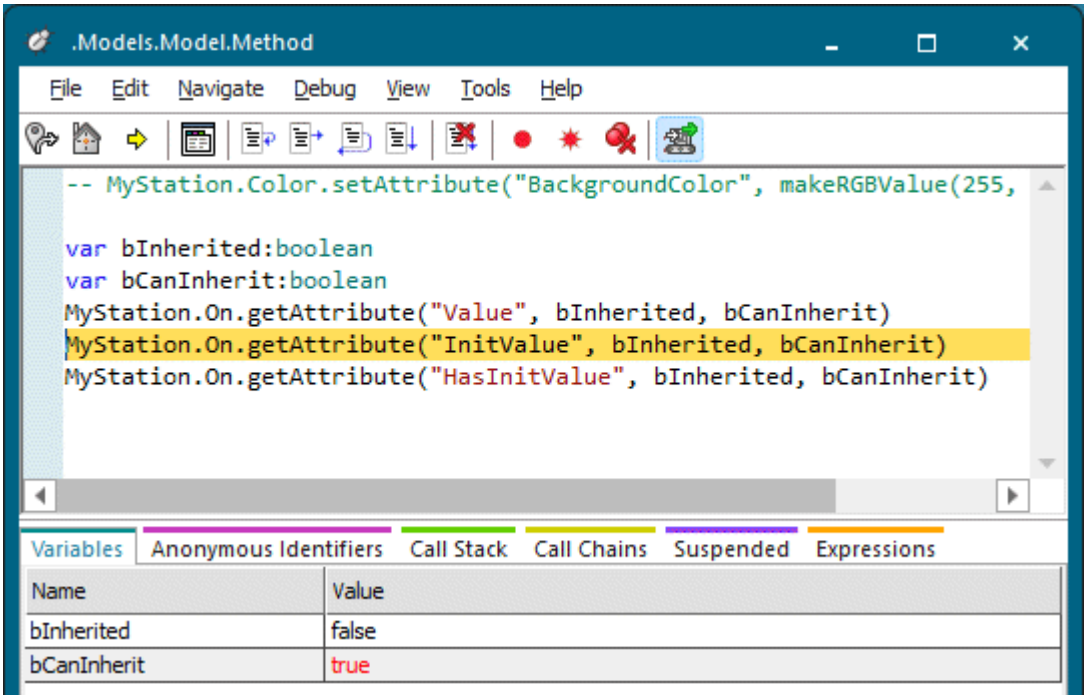
  In case an attribute cannot be inherited, the optional parameter *Inherited* always returns false.
- If a local variable of data type *boolean* is passed to the parameter *CanInherit*, *Plant Simulation* assigns true to this variable if the value of the attribute designated by string can be inherited, false if it cannot be inherited.

Return Value

The return value has the data type *any*.

Example

```
var bInherited:boolean
var bCanInherit:boolean
MyStation.On.getAttribute("Value", bInherited, bCanInherit)
MyStation.On.getAttribute("InitValue", bInherited, bCanInherit)
MyStation.On.getAttribute("HasInitValue", bInherited, bCanInherit)
```



SimTalk

setAttribute [SimTalk] - user-defined attribute

See also

User-defined Attributes [general description]

Create a User-defined Attribute During the Simulation

getAttrName [SimTalk]

Returns the name of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.getAttrName(AttributeNumber:integer) → string
```

Parameter

The parameter *AttributeNumber* of data type *integer* designates the name of the *user-defined attribute*. *Plant Simulation* shows an error message if the allowed index range is exceeded.

Return Value

The return value has the data type *string*.

Example

```
// lists all user-defined attributes of the object
for var index := 1 to MyStation.NumAttr
   print MyStation.getAttrName(index)
next
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

getAttrNo [SimTalk]

Returns the index number of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.getAttrNo(AttributeName:string) → integer
```

Parameter

The parameter *AttributeName* of data type *string* designates the name of the *user-defined attribute*.

Return Value

The return value has the data type *integer*.

0 if no *user-defined attribute* with the specified *AttributeName* exists.

Examples

```
param lookFor: string -> any
// checks if a user-defined object exists or not and
// returns its value if it exists
var index: integer := MyStation.getAttrNo(lookFor)
if index > 0
        return MyStation.getAttrValue(index)
end
```

```
param lookFor: string -> boolean
// checks if a user-defined object exists or not
var index: integer
index := MyStation.getAttrNo(lookFor)
if index = 0
   result := false // does not exist
else
    result := true // exists
end
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

getAttrType [SimTalk]

Returns the data type of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.getAttrType(AttributeNumber:integer) → string
```

Parameter

The parameter *AttributeNumber* of data type *integer* designates the *user-defined attribute*.

Return Value

The return value has the data type *string*.

Example

```
// lists the data types of all user-defined attributes
for var index := 1 to MyStation.NumAttr
    print MyStation.getAttrType(index)
next
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

getAttrValue [SimTalk]

Returns the value of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.getAttrValue(AttributeNumber:integer) → any
```

Parameter

The parameter *AttributeNumber* of data type *integer* designates the *user-defined attribute*. The data type of the value depends on the type of the attribute.

Return Value

The return value has the data type *any*.

Example

```
print MyStation.getAttrValue(1)
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

inheritAttribute [SimTalk] - user-defined attribute

Turns inheritance of the attribute designated by *AttributeName* of the *user-defined attribute* of the object designated by <Path> **on** if it was turned off.

Remarks

If several attributes are inherited together, all attributes will be inherited, when you turn inheritance of a single attribute on.

Type

Method

Syntax

```
<Path>.<UserDefinedAttribute>.inheritAttribute(AttributeName:string)
```

Parameter

The parameter *AttributeName* of data type *string* designates the name of the attribute of the *user-defined attribute*.

Examples

```
Station.MyInheritedAttribute.inheritAttribute("Transparent")
```

```
MyStation.MyAttribute.inheritAttribute("InitValue")
```

```
MyStation.MyAttribute.inheritAttribute("HasInitValue")
```

See also

User-defined Attributes [general description]

Create a User-defined Attribute During the Simulation

setAttribute [SimTalk] - user-defined attribute

Sets the value of the specified attribute of the *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.<UserDefinedAttribute>.setAttribute(AttributeName:string, Value:any)
```

Parameters

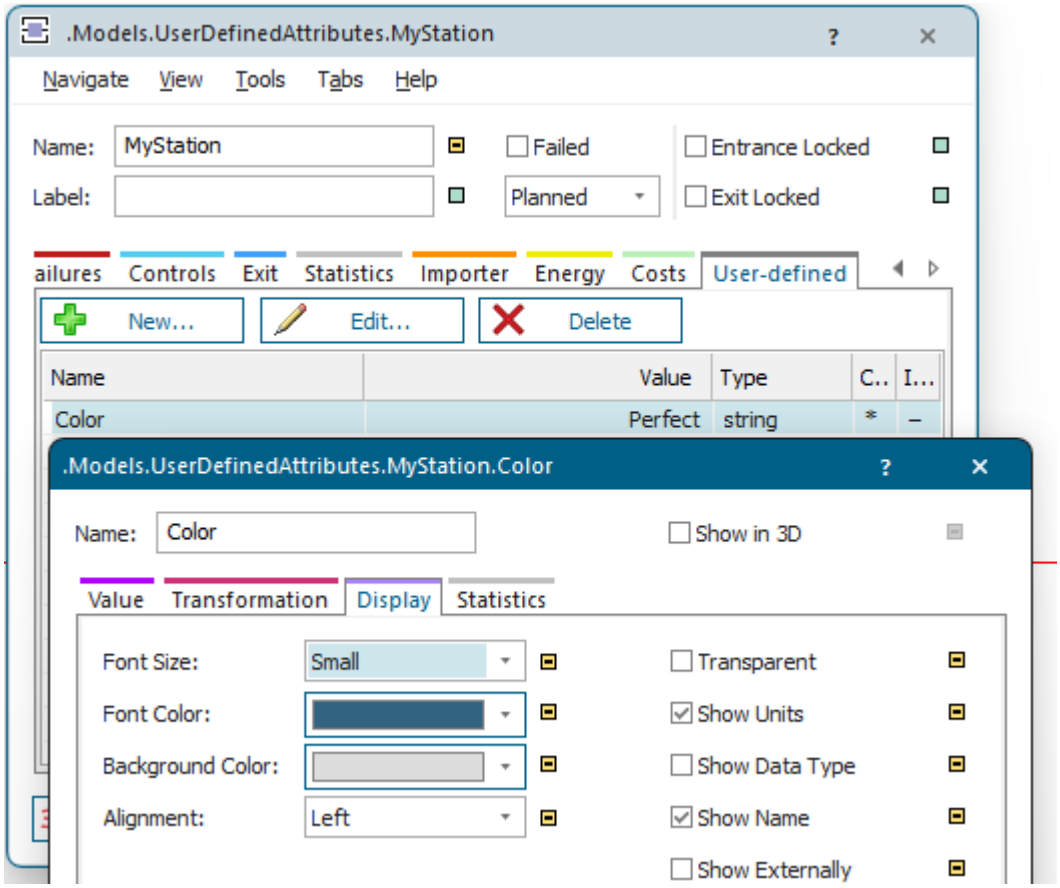
You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the name of the attribute of the *user-defined attribute*.
- The parameter *Value* of data type *any* designates the value of the *user-defined attribute*.

Example

```
MyStation.Color.setAttribute("BackgroundColor", makeRGBValue(255, 87, 192))
```

The source code sets the **Background Color** of the *user-defined attribute* named `Color`.



SimTalk

getAttribute [SimTalk] - user-defined attribute

See also

User-defined Attributes [general description]

Create a User-defined Attribute During the Simulation

setAttrType [SimTalk]

Sets the data type of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.setAttrType(AttributeNumber:integer, DataType:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *AttributeNumber* of data type *integer* designates the index number of the *user-defined attribute*.
- The parameter *DataType* of data type *string* designates its data type: *boolean*, *integer*, *real*, *string*, *object*, *table*, *list*, *stack*, *queue*, *time*, *money*, *length*, *weight*, *speed*, *acceleration*, *date*, *dateTime*, or *randtime*.

Return Value

The return value has the data type *boolean*.

Example

```
var index: integer
index := MyStore.getAttrNo("Sale")
MyStore.setAttrType(index,"boolean")
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

setAttrValue [SimTalk]

Sets the value of the designated *user-defined attribute* of the object designated by <Path>.

Type

Method

Syntax

```
<Path>.setAttrValue(AttributeNumber:integer, Value:any) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *AttributeNumber* of data type *integer* designates the index number of the *user-defined attribute*.
- The parameter *Value* of data type *any* designates its value.

Return Value

The return value has the data type *boolean*.

Example

```
var index: integer
index := MyStore.getAttrNo("Sale")
MyStore.setAttrValue(index,true)
```

See also

User-defined Attributes [general description]

New [user-defined attribute]

unshare [SimTalk]

Makes the *user-defined attribute* or *Variable* of the object designated by <Path> unshare a shared list and receive its own copy of the list.

Remarks

Unsharing might be required after assigning a *user-defined attribute* or a *Variable*, that has the data type *table/list/stack/queue*, to another *user-defined attribute* of the same data type. Then both attributes are independent of each other instead of using the same list.

It might also be required when *user-defined attributes* of data type *table/list/stack/queue*, are created when producing *MUs* with a **Delivery Table** with a *Source*.

When assigning a *user-defined attribute* of data type *table/list/stack/queue* to another *user-defined attribute*, *Plant Simulation* does not copy the list, but creates a reference to that list. Both *user-defined attributes* then share the same list, i.e., if one of the two *user-defined attributes* is changed, this also changes the contents of the other one.

If you do not want this behavior, use the method *unshare* for one of the two *user-defined attributes*. This results in the respective *user-defined attribute* having its own copy of the list. This way both *user-defined attributes* are independent of each other.

> **Note**
> This also applies to objects of type **Variable** of data type *table/list/stack/queue*.

Type

Method

Syntax

```
<Path>.unshare
```

Example

```
Station.TabAttr := ParallelStation.TabAttr
Station.TabAttr[1,1] := "Alice"
// share the user-defined table attributes
ParallelStation.TabAttr[1,1] := "Bob"
// change the value in the common table
print Station.TabAttr[1,1]
// prints Bob

ParallelStation.TabAttr.unshare
// now both attributes have their own table
```

```
ParallelStation.TabAttr[1,1] := "Charlie"
print Station.TabAttr[1,1]
// prints Bob
```

See also

Sequence Cyclical [MU selection]

Sequence [MU selection]

Random [MU selection]

Percentage [MU selection]

Value [Variable] > Unshare

Variable [object]

Unshare a List or Data Table

Miscellaneous Methods of User-defined Attributes

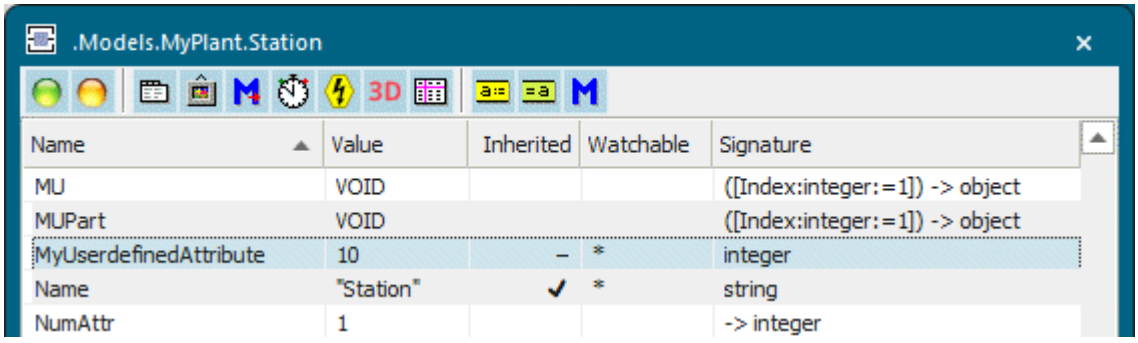
Miscellaneous Methods of User-defined Attributes

Most objects provide the *methods* listed in the table of contents to the left for the *user-defined attributes* themselves:

getStatisticsTable

increment [SimTalk]

The window **Show Attributes and Methods** shows the *user-defined attribute* itself and its value.



See also

User-defined Attributes [general description]

getStatisticsTable [SimTalk] - user-defined attribute

Returns the statistics table of the *user-defined attribute* of the object designated by <Path> and writes it into a table.

Remarks

If the *user-defined attribute* is of data type *string*, *Plant Simulation* always lower-cases the string values in the first column.

Type

Method

Syntax

```
<Path>.<UserDefinedAttribute>.getStatisticsTable(TargetTable:table)
```

Parameter

The parameter *TargetTable* of data type *table* designates the name of the table.

Example

```
MyStation.MyAttribute.getStatisticsTable(MyStatisticstable)
```

See also

User-defined Attributes [general description]

Statistics Table [user-defined attribute]

increment [SimTalk] - user-defined attribute

Adds 1 (the number 1) to the *user-defined attribute* of the object designated by <Path>.

Remarks

The method only applies to numerical data types. It applies to *user-defined attributes* and to the object *Variable*.

Type

Method

Syntax

```
<Path>.<UserDefinedAttribute>.increment
<Path>.<UserDefinedAttribute>.increment(Number:integer)
<Path>.<UserDefinedAttribute>.increment(Number:real)
```

Parameter

The parameter *Number* of data type *integer* or *real* designates the value that is added to the *user-defined attribute*.

Example

```
MyStation.MyAttribute.increment
// accomplishes the same as MyStation.attrib := MyStation.MyAttribute + 1
MyStation.MyAttribute.increment(-1)
// accomplishes the same as MyStation.MyAttribute := MyStation.MyAttribute
- 1
```

See also

User-defined Attributes [general description]

Read-Only Attributes of All Objects

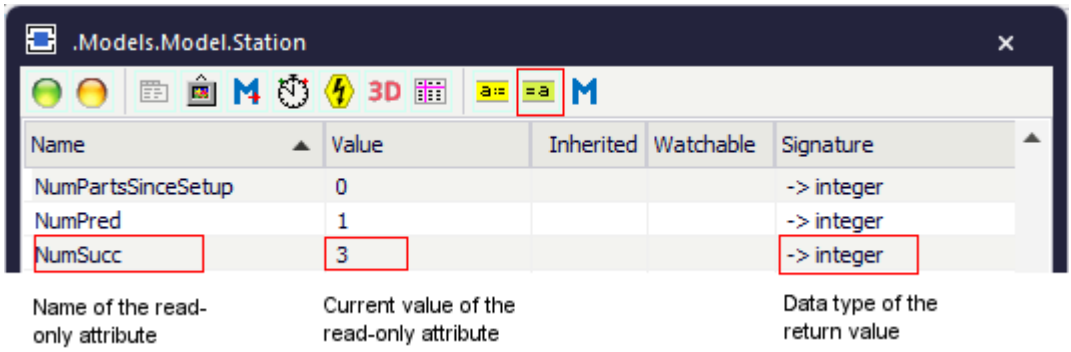
_Read-Only Attributes of All Objects

The **simulation objects** and the **3D objects** provide the *read-only attributes* listed in the table of contents to the left.

The objects in the *Class Library* share the *read-only attributes* described below.

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

To query the value of a *read-only attribute*, you might, for example, type:

```
print Source.Empty
```